

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №1

«Связи и запросы вариант 21 А»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Реализовать обработку связанных коллекций на Python. Вариант предметной области 21: Оператор и Язык программирования. Используется набор запросов А. Программа выполняет соединения один ко многим и многие ко многим, сортировку и агрегирование.

Модель данных

Operator хранит id, name, arity и language_id. Арность arity означает число операндов и используется как количественный признак. Language хранит id и name. OperatorLanguage содержит operator_id и language_id и реализует отдельную связь M:N. Основной язык в Operator задаёт учебное распределение для 1:N и не обозначает происхождение оператора.

Созданы 4 оператора (+, *, not, -), 3 языка (Python, Rust, Ada) и 5 записей связи. Оператор + связан и с Python, и с Rust; его основной язык в 1:N — Python. Все внешние ключи соответствуют существующим записям.

Перенос запросов на предметную область

A1 выводит пары операторов и основных языков с сортировкой по названию языка. При совпадении языка используется название оператора. A2 суммирует арности по основному языку и сортирует суммы по убыванию; при равенстве используется название языка. Пустые группы исключены, как в примере преподавателя.

A3 выбирает языки, содержащие отдельное слово «язык», и выводит списки связанных операторов через таблицу M:N. В шаблоне условия «сотрудник», «отдел», «зарплата» и слово «отдел» заменены соответственно оператором, языком, арностью и словом «язык».

Реализация и запуск

Для соединений применены list comprehensions, для A3 — dict comprehension. Используются sorted, sum, any и itemgetter. Данные заданы в памяти, внешние библиотеки и подключение к сети не требуются. Python 3.10 или новее.

```
python3 rk_01/main.py
```

Команда выполняется из корня pcpl-labs.

Результаты выполнения

```
$ python3 rk_01/main.py
РК-1. Вариант 21. Запросы А
А1. Операторы и языки, сортировка по языкам
('not', 1, 'язык Ada')
('*', 2, 'язык Python')
('+', 2, 'язык Python')
('-', 2, 'язык Rust')

А2. Суммарная арность по языкам, по убыванию
('язык Python', 4)
('язык Rust', 2)
('язык Ada', 1)

А3. Слово 'язык' в названии, связь M:N
язык Python: +, *
язык Rust: +, -
язык Ada: not
```

Рисунок 1. Экранная форма фактического вывода программы

Изображение построено по сохранённому stdout и stderr. Полные журналы приведены в папке results.

Проверка и вывод

Суммы арностей: Python — 4, Rust — 2, Ada — 1. В А3 для Rust выводятся + и -, что подтверждает использование связи M:N. Все три результата сопоставлены с функциями рефакторинга РК-2 скриптом tools/verify_extra.py; расхождений нет.

Источник задания

Условия РК-1, таблица вариантов, строка 21 и запросы А. Проверено 19 октября 2026 года.

https://github.com/ugapanyuk/courses_current/blob/main/python/rk/pcpl/pcpl_rk_1.pdf

Текст программы

```
# Файл main.py

"""РК-1. Вариант 21, запросы А. Запуск: python3 rk_01/main.py."""

from operator import itemgetter


class Operator:
    """Оператор: ключ, название, число операндов, основной язык."""

    def __init__(self, id, name, arity, language_id):
        self.id = id
        self.name = name
        self.arity = arity
        self.language_id = language_id


class Language:
    """Язык программирования на стороне «один»."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class OperatorLanguage:
    """Связь многие-ко-многим по идентификаторам."""

    def __init__(self, operator_id, language_id):
        self.operator_id = operator_id
        self.language_id = language_id


def main():
    # Учебные данные: арность означает число операндов оператора.
    # Основной язык нужен для связи 1:N, а не обозначает происхождение.
    languages = [
        Language(1, "язык Python"),
        Language(2, "язык Rust"),
        Language(3, "язык Ada"),
    ]
    operators = [
        Operator(1, "+", 2, 1),
        Operator(2, "*", 2, 1),
        Operator(3, "not", 1, 3),
        Operator(4, "-", 2, 2),
    ]
    links = [
```

Текст программы Продолжение

```
        OperatorLanguage(1, 1),
        OperatorLanguage(1, 2),
        OperatorLanguage(2, 1),
        OperatorLanguage(3, 3),
        OperatorLanguage(4, 2),
    ]

    one_to_many = [
        (op.name, op.arity, lang.name)
        for lang in languages
        for op in operators
        if op.language_id == lang.id
    ]
    many_to_many = [
        (op.name, op.arity, lang.name)
        for link in links
        for op in operators
        for lang in languages
        if op.id == link.operator_id and lang.id == link.language_id
    ]

    print("РК-1. Вариант 21. Запросы А")
    print("А1. Операторы и языки, сортировка по языкам")
    for row in sorted(one_to_many, key=itemgetter(2, 0)):
        print(row)

    print("\nА2. Суммарная арность по языкам, по убыванию")
    totals = [
        (lang.name, sum(row[1] for row in one_to_many
                        if row[2] == lang.name))
        for lang in languages
        if any(row[2] == lang.name for row in one_to_many)
    ]
    for row in sorted(totals, key=lambda row: (-row[1], row[0])):
        print(row)

    print("\nА3. Слово 'язык' в названии, связь M:N")
    result = {
        lang.name: [name for name, _, language in many_to_many
                    if language == lang.name]
        for lang in languages
        if "язык" in lang.name.casefold().split()
    }
    for language, names in result.items():
        print(f"{language}: {', '.join(names)}")


if __name__ == "__main__":
    main()
```